

Universidad de Antioquia

Informática II
2026-2

Desafío I

Sweet Crush mediante manipulación de bits en C++

Informe final

Juan Martín López González
Juan Camilo David Zapata

18 de Septiembre de 2026

Resumen

Este informe documenta el análisis, diseño y construcción de Sweet Crush, un juego escrito completamente en C++ con un tablero almacenado de forma comprimida a tres bits por casilla y con capacidad de crecer y encoger en el tiempo de ejecución mediante el uso de memoria dinámica.

1. Análisis del problema y consideraciones

1.1. El problema

Se pidió construir un juego de eliminación de fichas sobre una cuadrícula de dimensiones definidas por el usuario. Al eliminar una ficha las que están encima deben caer por gravedad y la parte superior se debe rellenar con fichas nuevas aleatorias y si al caer se forman alineaciones de tres o más fichas iguales esas alineaciones también deben desaparecer, lo que puede provocar caídas en cadena. Además, el tablero no es fijo, el usuario puede insertar o eliminar filas y columnas durante la partida.

Consideraciones obligatorias:

- Cada ficha debe almacenarse en exactamente 3 bits.
- El tablero debe estar en la memoria dinámica y adaptarse a los cambios de tamaño.
- Solo se permite el uso de arreglos, punteros, funciones y operadores de bits.
- La aplicación debe funcionar enteramente en la consola.

1.2. Almacenamiento considerado

El cálculo del tamaño total en bytes del tablero se definió mediante la siguiente expresión:

$$bytes = \left\lceil \frac{F \cdot C \cdot 3}{8} \right\rceil = \frac{F \cdot C \cdot 3 + 7}{8} \quad (1)$$

La suma de siete antes de la división entera es la forma de redondear hacia arriba por si los bits no completan un byte exacto, se reserva el byte parcial de todas formas para no perder información.

1.3. Consideraciones tomadas antes de programar

1. **Índice lineal:** La memoria es lineal. Se adoptó un orden por filas con una única función central para obtener posiciones (`fila * columnas + columna`) la dirección queda concentrada en un solo lugar por si a futuro se cambiara a orden por columnas, solo se modificaría esa función.
2. **Separar la dimensión:** El programa rastrea dos datos distintos: las filas y columnas (el tamaño que el jugador ve) y la `capacidad.bytes` (el tamaño real reservado en el heap). Esto permite evitar la reserva y liberación constante de memoria en cada operación de la estructura del tamaño.

3. **Crecimiento o liberación:** Se reserva un bloque de memoria nuevo solo cuando los bytes requeridos superan la capacidad actual y se libera solo cuando lo requerido baja del 65 % de la capacidad actual, sino tenemos esta función un usuario que agregara y eliminara una fila alternadamente provocaría operaciones costosas.
4. **Arquitectura Modular:** Se separó el código en unidades con responsabilidades diferentes (`main`, `menu`, `pantalla`, `tablero`, `juego` y `configuracion`), de modo que las capas no sepan nada y trabajen parcialmente independiente y no interacten directamente con la consola.

2. Esquema de tareas definidas para el desarrollo

El problema se descompuso en tareas con una entrada y una salida definidas. Tuvi-
mos como criterio principal que cada tarea pudiera probarse de forma independiente sin
que las demás estuvieran terminadas, las responsabilidades se dividieron en cinco áreas:
Almacenamiento, Reglas del juego, Dinamismo, Presentación y Control.

Área	Entrada	Salida	Función Responsable
T1. Almacenamiento	Filas, columnas	Bits y bytes requeridos	<code>calcular_bytes</code>
T1. Escritura	Arreglo, índice, valor	Arreglo modificado	<code>establecer_ficha</code>
T1. Lectura	Arreglo, índice	Valor numérico de 0 a 7	<code>obtener_ficha</code>
T2. Combinaciones	Tablero lógico	Marcas y # de combinaciones	<code>marcar_combinaciones</code>
T2. Gravedad	Tablero con huecos	Tablero compactado	<code>aplicar_gravedad</code>
T2. Cascadas	Tablero inestable	Tablero estable y contadores	<code>resolver_cascadas</code>
T3. Dinamismo	Posición estructural	Tablero ampliado/reducido	<code>agregar_fila</code> , <code>eliminar_fila</code>
T4. Presentación	Tablero y contadores	Vista en símbolos y binario	<code>mostrar_juego</code>
T5. Control	Interacción del usuario	Opciones y flujo principal	<code>main</code> , <code>mostrar_menu</code>

Cuadro 1: Desglose de tareas, entradas, salidas y responsables.

2.1. Flujo de un movimiento

El ciclo principal del juego opera bajo el siguiente flujo secuencial:

1. Se muestra el menú al usuario.
2. El usuario selecciona una opción válida (1 al 6).
3. Dependiendo de la opción (eliminar ficha, agregar/eliminar fila o columna), se modifica el estado o la estructura del tablero.
4. Se aplica la gravedad y se rellenan los huecos generados.

5. El sistema evalúa si existen combinaciones de 3 o más fichas.
6. Si existen combinaciones se eliminan, se acumula el puntaje y se retorna automáticamente al paso 4 (reacción en cadena).
7. Si no existen combinaciones, se muestra el tablero actualizado y las estadísticas, lo cual nos retorna al paso 1.

3. Algoritmos implementados

3.1. Escritura de una ficha de tres bits

El propósito es guardar un valor (de 0 a 7) en una posición exacta sin alterar las fichas vecinas, la ficha empieza en el bit $3p$, que se ubica físicamente en el byte $\lfloor 3p/8 \rfloor$, desplazado $3p \pmod{8}$ posiciones, si el desplazamiento es menor o igual a 5, los 3 bits caben dentro del mismo byte. Si es 6 o 7, la ficha queda partida y se debe trabajar sobre dos bytes.

La secuencia lógica consiste en limpiar primero el espacio de destino con una máscara negada (AND) y luego inyectar el valor nuevo usando una compuerta OR, esto garantiza la protección de los bits adyacentes (fichas vecinas), para el caso de frontera se utiliza una ventana temporal de 16 bits (`unsigned int`) para realizar la modificación de forma atómica.

3.2. Lectura de una ficha

Es la operación inversa a la escritura. Primero se desplazan los bits hacia la derecha y luego se enmascaran descartando así tanto los bits de la ficha anterior como los de la siguiente, el caso de cruce de dos bytes ocurre con desplazamientos de 6 o 7, ya que el cálculo de la capacidad redondea hacia arriba, siempre nos garantiza que el segundo byte necesario se encuentre dentro del bloque de memoria reservado, previniendo lecturas fuera de los límites.

3.3. Detección de combinaciones

Su propósito es ubicar grupos de tres o más fichas iguales, tanto horizontales como verticales sin duplicar el conteo de una misma casilla. Se recorre cada fila con dos punteros (inicio y fin) y Cuando se detecta una racha en lugar de eliminarla de inmediato registramos su posición en un arreglo booleano paralelo (marcadas), posteriormente se repite el recorrido por columnas.

Esta decisión es vital ya que si se borrarán los datos durante el barrido horizontal las figuras en L o en T perderían su intersección y el barrido vertical no las podría detectar.

3.4. Gravedad y relleno

Su función es compactar cada columna hacia abajo tras una eliminación y rellenar los espacios superiores vacíos, el algoritmo elimina las posiciones marcadas y desplaza las fichas restantes hacia abajo del tablero y genera fichas aleatorias válidas en la parte superior para restaurar las dimensiones de la cuadrícula.

3.5. Cascadas

Controla las reacciones en cadena llevando el tablero a un estado estable (sin combinaciones), consiste en un bucle que repite las tareas de detectar, eliminar y rellenar. Este ciclo termina utilizando el criterio de punto fijo, solo se detiene únicamente cuando la función de detección devuelve un valor de cero, además, Cada cascada incrementa exponencialmente el valor de la jugada.

3.6. Operaciones estructurales

Las cuatro funciones que cambian la estructura dimensional (agregar/eliminar filas o columnas) comparten un patrón de cuatro pasos:

1. Validar la coordenada objetivo y evitar colapsos (ej. eliminar la última fila restante).
2. Calcular la nueva exigencia de memoria y reservar un nuevo bloque en el heap si es necesario.
3. Transferir el estado lógico de las fichas al nuevo esquema.
4. Confirmar el reemplazo adoptando el nuevo bloque o reciclando el actual según la regla del 65 % de eficiencia.

3.7. Presentaciones simultaneas

La pantalla expone dos tableros simultáneos: la representación binaria de las fichas y la representación mediante caracteres simbólicos (@, #, \$, X, 8, A). Esto permite ver visualmente que el empaquetado y desplazamiento de 3 bits opera correctamente.

Problemas de desarrollo que se afrontaron

- **P1. Combinaciones contadas dos veces o perdidas:** Al principio con figuras en L o T el contador se desfasaba y algunas rachas verticales no se eliminaban.
Causa: Eliminar las fichas al instante alteraba el tablero antes de la segunda lectura.
Solución: Separar la etapa de detección de la etapa de eliminación usando el arreglo de marcas.
- **P2. Huecos atrapados en medio de la columna:** Tras eliminar fichas algunas celdas vacías no subían quedando atrapadas bajo piezas sólidas.
Causa: Recorrer la columna de arriba hacia abajo sobrescribía datos no leídos.
Solución: Invertir el sentido del recorrido (de abajo hacia arriba) usando índices independientes de lectura y escritura.
- **P3. ¿Cuándo termina una cascada?:** Debido al relleno aleatorio, se complicaba predecir cuántas reacciones en cadena ocurrirían.
Solución: Implementar un ciclo `while(true)` que se rompa exclusivamente al registrar 0 nuevas combinaciones.
- **P4. Alineación visual con dos cifras:** Al expandir el tablero a más de 10 casillas los encabezados desplazaban la cuadrícula.
Solución: Implementar relleno dinámico de espacios basado en el número de dígitos.

4. Evolución de la solución y consideraciones

El proyecto se desarrolló de forma iterativa:

1. **Prototipo:** Se construyó la lógica del juego (gravedad, combinaciones, cascadas) sobre una estructura tradicional, sin el uso de bits, para validar que teníamos correcto el algoritmo central.
2. **Empaquetado:** Una vez correcta la lógica, se reemplazó el acceso directo a los arreglos por las funciones (`obtener_ficha` y `establecer_ficha`), permitiendo integrar la arquitectura de 3 bits sin romper el juego.
3. **Separación por Módulos:** El código se segmentó tempranamente en múltiples archivos (`.cpp` y `.h`) para facilitar la depuración y así aislar errores.
4. **Estadísticas:** Como último se integró el control de límites, validación de entrada por consola y la puntuación.

5. Conclusión

El proyecto cumplió con los requisitos estructurales y lógicos establecidos. Por otro lado, se logró materializar un tablero virtual de dimensiones completamente dinámicas cuyo estado físico opera sobre un volumen continuo de bytes, demostrando que se pudo lograr el cometido de manipulación de datos a nivel de hardware.